

Módulo 1: Fundamentos do Expo

Lição 1.1: O Ecossistema Expo vs. React Native "Puro"

Explicação: No desenvolvimento mobile tradicional com React Native (chamado de *CLI*), você precisa configurar todo o ambiente nativo na sua máquina: Android Studio (para Android) e Xcode (para iOS). Isso exige muito processamento e tempo de configuração.

O **Expo** funciona como uma camada de abstração sobre o React Native. Ele oferece um conjunto de ferramentas e serviços que eliminam a necessidade de lidar com código nativo (Java/Kotlin ou Swift/Obj-C) no início do projeto.

- **Expo Go:** É um aplicativo que você instala no seu celular real. Ele lê o código JavaScript do seu computador via Wi-Fi e renderiza o app instantaneamente, permitindo que você teste no dispositivo físico sem cabos ou configurações complexas.
-

Lição 1.2: Componentes Primitivos (**View** e **Text**)

Explicação: Diferente do desenvolvimento Web, onde o navegador entende tags como `<h1>`, `<p>` ou `<div>`, o processador do celular não sabe o que é um HTML. Ele entende componentes nativos.

- **<View>:** Imagine que a **View** é uma caixa de plástico invisível. Ela é o componente mais fundamental. No Android, ela se transforma em uma `android.view`, e no iOS, em uma `UIView`. Você a usa para agrupar outros elementos, criar margens e definir o layout da tela. Sem uma **View** pai, você não consegue organizar múltiplos elementos em uma tela.
 - **<Text>:** No mobile, o sistema é rigoroso. Qualquer caractere de texto **precisa** estar dentro de um componente `<Text>`. Se você colocar uma palavra solta dentro de uma **View**, o app irá quebrar. Além disso, estilos de fonte (tamanho, cor, peso) devem ser aplicados diretamente no componente de texto.
-

Lição 1.3: Estilização com **StyleSheet** e Flexbox

Explicação: No Expo, não usamos arquivos `.css` externos. A estilização é feita através de objetos JavaScript usando a API `StyleSheet`.

Diferenças Cruciais:

1. **CamelCase:** No CSS usamos `background-color`; no Expo usamos `backgroundColor`.
2. **Flexbox por Padrão:** Na Web, o padrão de uma `div` é `display: block`. No mobile, toda `View` já nasce com `display: flex`.
3. **Direção do Eixo:** Na Web, o `flexDirection` padrão é `row` (linha). No mobile, o padrão é `column` (coluna), pois as telas de celular são usadas verticalmente.
4. **Unidades:** Não usamos `px`, `rem` ou `%` para tudo. Usamos números puros que representam "pontos de densidade independente", garantindo que o botão tenha o mesmo tamanho visual em um iPhone pequeno ou em um Android gigante.

QUESTIONÁRIO MODULO 1

1. Qual componente é obrigatório utilizar para exibir qualquer tipo de texto no Expo/React Native?

- A) `<View>`
- B) `<Text>`
- C) `<Label>`
- D) `<span>`

2. Qual é a principal função do componente `<View>` na construção de uma tela?

- A) Funcionar como um container para agrupar e organizar outros componentes.
- B) Serve apenas para exibir imagens de fundo.
- C) Realizar cálculos matemáticos e lógica de programação.
- D) Criar links diretos para sites externos.

3. Sobre a estilização (StyleSheet), qual é o padrão de escrita das propriedades no Expo?

- A) `kebab-case` (ex: `background-color`)
- B) `snake_case` (ex: `background_color`)
- C) `camelCase` (ex: `backgroundColor`)
- D) `UPPERCASE` (ex: `BACKGROUND_COLOR`)

4. O que acontece se um desenvolvedor tentar renderizar um texto solto diretamente dentro de uma `<View>`?

- A) O texto aparece com a cor padrão preta.
- B) O texto é ignorado e não aparece na tela.
- C) O aplicativo sofre um erro crítico (crash) e para de funcionar.
- D) O texto é convertido automaticamente em um botão.

5. Qual é o comportamento de alinhamento padrão (`flexDirection`) de uma `<View>` no ambiente mobile?

- A) `row` (os itens ficam um ao lado do outro, em linha).
- B) `column` (os itens ficam um abaixo do outro, em coluna).
- C) `grid` (os itens se organizam em quadrados automáticos).
- D) `diagonal` (os itens seguem o eixo inclinado da tela).



Módulo 2: Mão na Massa – Criando seu Primeiro Projeto

Lição 2.1: O Comando Global e o Node.js

Explicação: Antes de digitar qualquer código, o seu computador precisa entender a linguagem JavaScript fora do navegador. Para isso, instalamos o **Node.js**. Com ele, ganhamos o **NPM** (Node Package Manager), que é como uma loja de aplicativos, mas para bibliotecas de código.

No terminal, o comando `npx` é o nosso melhor amigo. Ele permite que executemos ferramentas sem precisar instalá-las permanentemente no computador, garantindo que você sempre use a versão mais recente do Expo.

Lição 2.2: Criando a Estrutura (O comando `create-expo-app`)

Explicação: Para criar um projeto do zero com a estrutura moderna (usando o sistema de roteamento mais novo, o **Expo Router**), utilizamos um único comando.

No seu terminal, digite:

Bash

```
npx create-expo-app@latest meu-app-gamificado
```

- `npx`: Executa o instalador.
- `create-expo-app@latest`: É o nome do "molde" oficial do Expo.
- `meu-app-gamificado`: É o nome da pasta que será criada. Você pode dar o nome que quiser (sem espaços ou acentos).

Após o comando, o terminal vai baixar todas as bibliotecas necessárias (como React e React Native) e configurar os arquivos de sistema que explicamos no Módulo 1.

Lição 2.3: Navegando e Iniciando o Servidor

Explicação: Com a pasta criada, você precisa "entrar" nela e dar a partida no motor do projeto. O comando `npx expo start` inicia o **Metro Bundler**, que é o servidor que compila seu código em tempo real.

Execute os comandos abaixo:

Bash

`cd meu-app-gamificado`

`npx expo start`

Ao rodar isso, um **QR Code** gigante aparecerá no seu terminal.

- **No Android:** Abra o app **Expo Go** e use o scanner de QR Code.
- **No iOS:** Abra a **Câmera padrão** e aponte para o código.

Lição 2.4: Comandos Úteis do Terminal

Explicação: Enquanto o servidor está rodando, você pode interagir com ele pressionando teclas específicas no teclado do seu computador:

- **r:** Recarrega o app (Reload). Útil se algo travar.
- **d:** Abre o Menu de Desenvolvedor no celular (onde você pode ver erros detalhados).
- **i:** Abre o simulador de iOS (se você estiver no Mac).
- **a:** Abre o emulador de Android.

questionário modulo 2

Questão 1 (Ordenação): Qual a ordem correta para criar e rodar um projeto?

1. `cd nome-do-app`
 2. `npx expo start`
 3. `npx create-expo-app@latest nome-do-app`
- **A) 1, 2, 3**
 - **B) 3, 1, 2 (CORRETA)**
 - **C) 2, 3, 1**

Questão 2 (Múltipla Escolha): Qual comando usamos para entrar na pasta do projeto via terminal?

- A) `go to folder`
- B) `npx enter`
- C) `cd` (Change Directory) (**CORRETA**)

Questão 3 (Verdadeiro ou Falso): O comando `npx expo start` apaga todo o meu código e reinicia o projeto do zero.

- () Verdadeiro
- () Falso (**CORRETA - Ele apenas inicia o servidor de desenvolvimento**)

Módulo 3: Interatividade e Dinâmica

Lição 3.1: O Hook `useState` (A Memória do App)

Explicação: Imagine que você está criando o botão de "curtir" do seu app. Quando o usuário toca no botão, o coração precisa mudar de cinza para vermelho. Como o app sabe que o estado mudou?

No Expo (React), usamos o `useState`. Ele é um "gancho" que permite ao componente "lembrar" de informações.

- Se você altera uma variável comum, a tela não atualiza.
- Se você altera um **Estado**, o Expo redesenha o componente instantaneamente para mostrar o novo valor.

É assim que o seu app de gamificação vai atualizar a barra de progresso ou a quantidade de XP na tela toda vez que o aluno acertar uma questão.

Lição 3.2: Capturando Toques com `TouchableOpacity`

Explicação: Na web, usamos a tag `<button>`. No Expo, o componente mais usado para interatividade é o `TouchableOpacity`.

O nome é autoexplicativo: "Touch" (Toque) + "Opacity" (Opacidade). Quando o usuário encosta o dedo no botão, ele fica levemente transparente por um milésimo de segundo. Esse efeito visual é vital no mobile para que o usuário sinta que o clique funcionou (o que chamamos de *feedback tátil*).

Lição 3.3: Props Dinâmicas

Explicação: Agora que sabemos usar o Estado, podemos passar esse estado para outros componentes via **Props**.

Por exemplo: você tem um componente chamado **CardDeLicao**. Você pode passar uma Prop chamada **bloqueado={true}**. Dentro do componente, você usa uma lógica simples: "Se bloqueado for verdadeiro, mostre um ícone de cadeado e mude a cor para cinza". Isso permite que uma única estrutura de código exiba diferentes estados do curso.

QUESTIONARIO MODULO 3

1. Para que serve o Hook **useState** no desenvolvimento com Expo?

- A) Para mudar a cor de fundo do aplicativo permanentemente.
- B) Para permitir que o componente armazene e reaja a mudanças de informações na tela.
- C) Para conectar o aplicativo diretamente com a loja da Apple ou Google.
- D) Para deletar arquivos temporários do celular do usuário.

2. Qual é a principal característica visual do componente **TouchableOpacity** ao ser tocado?

- A) Ele aumenta de tamanho e muda de posição na tela.
- B) Ele emite um som de clique obrigatório.
- C) Ele reduz levemente sua opacidade (fica mais transparente) para indicar o toque.
- D) Ele abre automaticamente o navegador de internet.

3. O que acontece com um componente quando o valor do seu Estado (**State**) é atualizado?

- A) O componente é excluído da memória.
- B) O componente é renderizado (redenhado) novamente para refletir a mudança.
- C) O celular do usuário precisa ser reiniciado.
- D) Nada acontece visualmente, apenas no código interno.

4. Em um app de ensino, qual seria o melhor uso para o **useState**?

- A) Guardar o nome do desenvolvedor do aplicativo.
- B) Controlar se a resposta de um exercício está correta para mostrar o feedback.
- C) Definir o tamanho fixo da imagem da logo.
- D) Instalar novas bibliotecas no projeto.

5. Qual a diferença fundamental entre uma Prop e um Estado (**State**)?

- A) Props são passadas de pai para filho (externas), enquanto o Estado é gerenciado dentro do próprio componente (interno).
- B) Props só aceitam números e o Estado só aceita textos.

- C) O Estado nunca muda, mas as Props mudam o tempo todo.
- D) Não existe diferença; são nomes diferentes para a mesma função.